Table of Contents

1. Overview
2. Authentication
   - Authentication Methods
   - CORS Policy
3. User Worker API
   - Endpoints
     - Get User Data
     - Create/Update User
     - Delete User
     - Add Cases to User
     - Remove Cases from User
4. Image Worker API
   - Endpoints
     - Upload Image
     - Get Signed URL
     - Delete Image
5. PDF Worker API
   - Endpoints
     - Generate PDF
6. Data Worker API
   - Endpoints
     - Get File Data
     - Save File Data
     - Delete File
     - Sign Forensic Manifest
     - Sign Confirmation Export
     - Sign Audit Export Metadata
7. Keys Worker API
   - Endpoints
     - Get Environment Key
8. Audit Worker API
   - Endpoints
     - Store Audit Entry
     - Get User Audit Entries
9. Error Handling
   - Standard Error Response Format
   - Common Error Codes
   - HTTP Status Codes
10. Type Definitions
    - Audit Trail Types
      - ValidationAuditEntry Interface
      - AuditAction Type
      - AuditResult Type
      - AuditDetails Interface
      - AuditTrail Interface
      - AuditSummary Interface
      - WorkflowPhase Type
    - Core Annotation Types
      - AnnotationData Interface
      - ConfirmationData Interface
      - BoxAnnotation Interface
    - Case Management Types
      - CaseData Interface
      - CaseActionType
      - CasesToDelete Interface
    - Case Export Types
      - CaseExportData Interface
      - AllCasesExportData Interface
      - ExportFormat Type
      - ExportOptions Interface

- Case Import Types
  - CaseImportProps Interface
  - ImportOptions Interface
  - ImportResult Interface
  - ConfirmationImportResult Interface
  - ReadOnlyCaseMetadata Interface
  - CaseImportPreview Interface
- File Management Types
  - FileData Interface
  - FileUploadResponse Interface
  - ImageUploadResponse Interface
- User Management Types
  - UserData Interface
  - UserLimits Interface
- Centralized Type Management

11. SDK Examples
    - JavaScript/TypeScript Client
    - Error Handling Example

Overview

Striae's backend consists of six Cloudflare Workers that provide specialized API services. All APIs use RESTful conventions and return JSON responses.

Authentication

Authentication Methods

All worker APIs use custom authentication headers:

```
X-Custom-Auth-Key: {api_key}
```

API keys are generated by the Keys Worker and should be obtained through the authentication flow.

Note: The Image Worker uses a different authentication method with `Authorization: Bearer {token}` header.

Authentication Environment Variables:

- **User Worker**: Uses `USER_DB_AUTH`, `R2_KEY_SECRET`, `IMAGES_API_TOKEN`, `PROJECT_ID`, `FIREBASE_SERVICE_ACCOUNT_EMAIL`, and `FIREBASE_SERVICE_ACCOUNT_PRIVATE_KEY`
- **Data Worker**: Uses `R2_KEY_SECRET`, `MANIFEST_SIGNING_PRIVATE_KEY`, and `MANIFEST_SIGNING_KEY_ID` plus `STRIAE_DATA` bucket binding
- **Audit Worker**: Uses `R2_KEY_SECRET` plus `STRIAE_AUDIT` bucket binding
- **Keys Worker**: Uses `KEYS_AUTH` environment variable
- **Image Worker**: Uses `API_TOKEN` environment variable
- **PDF Worker**: No authentication required

CORS Policy

All workers implement strict CORS policies:

- **Allowed Origin**: `https://app.striae.org`
- **Allowed Methods**: Varies by worker
- **Allowed Headers**: `Content-Type, X-Custom-Auth-Key, Authorization`

User Worker API

Base URL: `{USER_WORKER_URL}`

Authentication: `X-Custom-Auth-Key` header

Environment Variables Required: - `USER_DB_AUTH` - KV database authentication - `R2_KEY_SECRET` - Data worker API authentication for case deletion - `IMAGES_API_TOKEN` - Image worker API authentication for file deletion - `PROJECT_ID` - Firebase project ID for admin account deletion - `FIREBASE_SERVICE_ACCOUNT_EMAIL` - Service account client email for admin deletion token exchange - `FIREBASE_SERVICE_ACCOUNT_PRIVATE_KEY` - PKCS#8 private key for signing admin deletion JWT assertions

Endpoints

Get User Data

```
GET /{userUid}
```

Description: Retrieve user profile data from KV storage

Parameters:

- `userUid` (path): Firebase user UID

Response:

```
{
  "uid": "string",
  "email": "string",
  "firstName": "string",
  "lastName": "string",
  "company": "string",
  "permitted": boolean,
  "cases": [
    {
      "caseNumber": "string",
      "createdAt": "ISO 8601 date"
    }
  ],
  "createdAt": "ISO 8601 date",
  "updatedAt": "ISO 8601 date"
}
```

Status Codes:

- `200`: Success
- `404`: User not found
- `403`: Forbidden (invalid auth header)
- `500`: Server error

Create/Update User

```
PUT /{userUid}
```

Description: Create new user or update existing user data. Preserves existing cases when updating.

Parameters:

- `userUid` (path): Firebase user UID

Request Body:

```
{
  "email": "string",
  "firstName": "string",
  "lastName": "string",
  "company": "string",
  "permitted": boolean
}
```

Response: User object (same as GET response)

Status Codes:

- `200`: User updated
- `201`: User created
- `403`: Forbidden
- `500`: Server error

Delete User

```
DELETE /{userUid}
```

Description: Delete user and all associated data including cases, files, and images.

Streaming Option: Supports server-sent event progress when either: - `stream=true` is provided as a query parameter - `Accept: text/event-stream` is sent

Parameters:

- `userUid` (path): Firebase user UID

Process: 1. Retrieves user data 2. Deletes all user's cases and associated files from data worker 3. Deletes all associated images from image worker
4. Deletes user account from KV storage

Security Notes:

- Demo accounts (`permitted: false`) should be protected from deletion at the UI level
- Deletion includes all user data, cases, files, and annotations
- Action is irreversible and permanent

Response:

```
{
  "success": true,
  "message": "Account successfully deleted"
}
```

Error Response:

```
{
  "success": false,
  "message": "Account deletion failed"
}
```

Status Codes:

- `200`: Success
- `404`: User not found
- `403`: Forbidden
- `500`: Server error

Add Cases to User

```
PUT /{userUid}/cases
```

Description: Add case assignments to user. Filters out duplicate case numbers.

Parameters:

- `userUid` (path): Firebase user UID

Request Body:

```
{
  "cases": [
    {
      "caseNumber": "string"
    }
  ]
}
```

Response: Updated user object

Status Codes:

- `200`: Success
- `404`: User not found
- `403`: Forbidden
- `500`: Server error

Remove Cases from User

```
DELETE /{userUid}/cases
```

Description: Remove case assignments from user. Does not delete the actual case data.

Parameters:

- `userUid` (path): Firebase user UID

Request Body:

```
{
  "casesToDelete": ["caseNumber1", "caseNumber2"]
}
```

Response: Updated user object

Status Codes:

- `200`: Success
- `404`: User not found
- `403`: Forbidden
- `500`: Server error

Image Worker API

Base URL: `{IMAGE_WORKER_URL}`

Authentication: `Authorization: Bearer {API_TOKEN}` header

Endpoints

Upload Image

```
POST /
```

Description: Upload image to Cloudflare Images

Request: Multipart form data with image file

Response:

```
{
  "result": {
    "id": "string",
    "filename": "string",
    "uploaded": "ISO 8601 date",
    "requireSignedURLs": true,
    "variants": ["string"]
  },
  "success": boolean,
  "errors": [],
  "messages": []
}
```

Status Codes:

- `200`: Success
- `403`: Forbidden (invalid token)
- `400`: Invalid file or missing file
- `500`: Server error

Get Signed URL

```
GET /{imageDeliveryPath}
```

Description: Generate signed URL for image access from imagedelivery.net URL

Parameters:

- `imageDeliveryPath` (path): Full imagedelivery.net URL path

Response: Signed URL as plain text

Status Codes:

- `200`: Success
- `403`: Forbidden (invalid token)
- `500`: Server error

Delete Image

```
DELETE /{imageId}
```

Description: Delete image from Cloudflare Images

Parameters:

- `imageId` (path): Cloudflare Images ID

Response:

```
{
  "success": boolean,
  "result": {},
  "errors": [],
  "messages": []
}
```

Status Codes:

- `200`: Success
- `403`: Forbidden (invalid token)
- `404`: Image not found
- `500`: Server error

PDF Worker API

Base URL: `{PDF_WORKER_URL}`

Authentication: None required

Endpoints

Generate PDF

`POST /`

Description: Generate PDF report with annotations

Modular Report Formats: For architecture details and custom format module implementation, see PDF Report System.

Request Body (preferred):

```
{
  "reportFormat": "striae",
  "data": {
    "imageUrl": "string",
    "caseNumber": "string",
    "annotationData": {
      "leftCase": "string",
      "rightCase": "string",
      "leftItem": "string",
      "rightItem": "string",
      "caseFontColor": "string",
      "classType": "Bullet | Cartridge Case | Other",
      "customClass": "string",
      "classNote": "string",
      "indexType": "number | color",
      "indexNumber": "string",
      "indexColor": "string",
      "supportLevel": "ID | Exclusion | Inconclusive",
      "hasSubclass": boolean,
      "includeConfirmation": boolean,
      "additionalNotes": "string",
      "boxAnnotations": [
        {
          "id": "string",
          "x": "number (percentage 0-100)",
          "y": "number (percentage 0-100)",
          "width": "number (percentage 0-100)",
          "height": "number (percentage 0-100)",
          "color": "string (hex color)"
        }
      ]
    },
    "activeAnnotations": ["string"],
    "currentDate": "string",
    "notesUpdatedFormatted": "string",
    "userCompany": "string"
  }
}
```

Legacy compatibility: The worker also accepts legacy top-level payloads (without `reportFormat` and `data`) for backward compatibility.

Response: PDF file (binary data)

Headers: - `Content-Type`: `application/pdf`

Status Codes: - `200`: Success - `405`: Method not allowed - `500`: Server error

Data Worker API

Base URL: `{DATA_WORKER_URL}`

Authentication: `X-Custom-Auth-Key` header

Environment Variables Required: - `R2_KEY_SECRET` - Data worker authentication secret - `MANIFEST_SIGNING_PRIVATE_KEY` - PKCS#8 private key used for signing - `MANIFEST_SIGNING_KEY_ID` - Signing key identifier embedded in signature metadata - `STRIAE_DATA` - R2 bucket binding that stores case and annotation JSON payloads

Endpoints

Get File Data

```
GET /{filename}.json
```

Description: Retrieve JSON file data from R2 storage

Parameters: - `filename` (path): JSON filename (without .json extension in URL)

Response: JSON data from file or empty array if file doesn't exist

Status Codes: - `200`: Success - `403`: Forbidden - `500`: Server error

Save File Data

```
PUT /{filename}.json
```

Description: Save JSON data to R2 storage file

Parameters: - `filename` (path): JSON filename (without .json extension in URL)

Request Body: Any valid JSON data

Response:

```
{
  "success": true
}
```

Status Codes: - `200`: Success - `400`: Invalid file type (non-JSON) - `403`: Forbidden - `500`: Server error

Delete File

```
DELETE /{filename}.json
```

Description: Delete JSON file from R2 storage

Parameters: - `filename` (path): JSON filename (without .json extension in URL)

Response:

```
{
  "success": true
}
```

Status Codes: - `200`: Success - `404`: File not found - `403`: Forbidden - `500`: Server error

Sign Forensic Manifest

```
POST /api/forensic/sign-manifest
```

Description: Sign canonical forensic manifest payload with the server-side private key.

Request Body:

```
{
  "manifest": {
    "dataHash": "64-char sha256 hex",
    "imageHashes": {
      "image-1.jpg": "64-char sha256 hex"
    },
    "manifestHash": "64-char sha256 hex",
    "totalFiles": 2,
    "createdAt": "2026-03-12T00:00:00.000Z"
  }
}
```

Notes: - The payload may be provided either at `manifest` or at the JSON root. - Returns `manifestVersion: "2.0"` when successful.

Response:

```
{
  "success": true,
  "manifestVersion": "2.0",
  "signature": {
    "algorithm": "RSASSA-PKCS1-v1_5-SHA-256",
    "keyId": "forensic-signing-key-v1",
    "signedAt": "2026-03-12T00:00:00.000Z",
    "value": "base64url-signature"
  }
}
```

Status Codes: - `200`: Signature created - `400`: Invalid manifest payload - `403`: Forbidden - `500`: Server error

Sign Confirmation Export

```
POST /api/forensic/sign-confirmation
```

Description: Sign canonical confirmation export payload with the server-side private key.

Request Body:

```
{
  "confirmationData": {
    "metadata": {
      "caseNumber": "CASE-2026-001",
      "exportDate": "2026-03-12T00:00:00.000Z",
      "exportedBy": "reviewer@example.com",
      "exportedByUid": "uid-123",
      "exportedByName": "Reviewer Name",
      "exportedByCompany": "Agency",
      "totalConfirmations": 1,
      "version": "2.0",
      "hash": "64-char sha256 hex"
    },
    "confirmations": {
      "original-image-id": [
        {
          "fullName": "Reviewer Name",
          "badgeId": "ABC123",
          "timestamp": "03/12/2026 12:00:00",
          "confirmationId": "CONF-123",
          "confirmedBy": "uid-123",
          "confirmedByEmail": "reviewer@example.com",
          "confirmedByCompany": "Agency",
          "confirmedAt": "2026-03-12T00:00:00.000Z"
        }
      ]
    }
  },
  "signatureVersion": "2.0"
}
```

Notes: - `signatureVersion` defaults to `2.0` when omitted. - Only `2.0` is accepted. - The payload may be provided either at `confirmationData` or at the JSON root.

Response:

```
{
  "success": true,
  "signatureVersion": "2.0",
  "signature": {
    "algorithm": "RSASSA-PKCS1-v1_5-SHA-256",
    "keyId": "forensic-signing-key-v1",
    "signedAt": "2026-03-12T00:00:00.000Z",
    "value": "base64url-signature"
  }
}
```

Status Codes: - `200`: Signature created - `400`: Invalid confirmation payload or unsupported signature version - `403`: Forbidden - `500`: Server error

Sign Audit Export Metadata

```
POST /api/forensic/sign-audit-export
```

Description: Sign canonical audit export metadata with the server-side private key.

Request Body:

```
{
  "auditExport": {
    "signatureVersion": "1.0",
    "exportFormat": "json",
    "exportType": "trail",
    "scopeType": "case",
    "scopeIdentifier": "CASE-2026-001",
    "generatedAt": "2026-03-12T00:00:00.000Z",
    "totalEntries": 42,
    "hash": "64-char sha256 hex"
  },
  "signatureVersion": "1.0"
}
```

Notes: - `signatureVersion` defaults to `1.0` when omitted. - Only `1.0` is accepted. - The payload may be provided either at `auditExport` or at the JSON root.

Response:

```
{
  "success": true,
  "signatureVersion": "1.0",
  "signature": {
    "algorithm": "RSASSA-PKCS1-v1_5-SHA-256",
    "keyId": "forensic-signing-key-v1",
    "signedAt": "2026-03-12T00:00:00.000Z",
    "value": "base64url-signature"
  }
}
```

Status Codes: - `200`: Signature created - `400`: Invalid audit export payload or unsupported signature version - `403`: Forbidden - `500`: Server error

Note: Audit entry APIs are documented under the Audit Worker section below.

Keys Worker API

Base URL: `{KEYS_WORKER_URL}`

Authentication: `X-Custom-Auth-Key` header

Endpoints

Get Environment Key

`GET /{keyName}`

Description: Retrieve environment variable value

Parameters: - `keyName` (path): Environment variable name

Response: Plain text value of the environment variable

Status Codes: - `200`: Success - `400`: Key name required - `404`: Key not found - `403`: Forbidden - `500`: Server error

Audit Worker API

Base URL: `{AUDIT_WORKER_URL}`

Authentication: `X-Custom-Auth-Key` header with API key from Keys Worker

Storage: Cloudflare R2 bucket (`STRIAE_AUDIT` - Dedicated audit bucket)

Environment Variables Required: - `R2_KEY_SECRET` - R2 bucket authentication for accessing STRIAE_AUDIT bucket - `STRIAE_AUDIT` - R2 bucket binding for audit trail storage

Endpoints

Store Audit Entry

`POST /audit/?userId={userId}`

Description: Create a new audit trail entry for forensic accountability and compliance tracking

Parameters: - `userId` (query, required): User identifier for audit trail organization

Request Body:

```
{
  "timestamp": "2025-09-23T14:30:15.123Z",
  "userId": "aDzwq3G6IBVRJVCEFijdg7B0fwq2",
  "userEmail": "analyst@example.com",
  "action": "export",
  "result": "success",
  "fileName": "CASE-2025-001-audit.json",
  "details": {
    "caseNumber": "CASE-2025-001",
    "workflowPhase": "case-export",
    "validationErrors": []
  }
}
```

Validation Notes: - Required fields validated by the worker: `timestamp`, `userId`, and `action` - Additional fields are accepted and persisted as part of the audit payload

Response:

```
{
  "success": true,
  "entryCount": 42,
  "filename": "audit-trails/aDzwq3G6IBVRJVCEFijdg7B0fwq2/2025-09-23.json"
}
```

Status Codes: - `200`: Audit entry stored successfully - `400`: Invalid request data or missing required fields - `403`: Forbidden - `500`: Server error

Audit Entry Fields:

Field	Type	Required	Description
`timestamp`	string (ISO 8601)	Yes	When the action occurred
`userId`	string	Yes	User identifier for trail grouping
`action`	string	Yes	Action name (for example `case-create`, `export`, `import`)
`result`	string	No	Result status such as `success`, `failure`, `warning`, or `blocked`
`details`	object	No	Action-specific metadata and workflow context
`userEmail`	string	No	User email for analyst attribution
`fileName`	string	No	Associated file name
`caseNumber`	string	No	Case identifier when action is case-scoped

Get User Audit Entries

```
GET /audit/?userId={userId}
```

Description: Retrieve audit trail entries for a specific user with optional filtering

Query Parameters: - `userId` (required): User identifier - `startDate` (optional): Start date filter (YYYY-MM-DD or ISO 8601 format) - `endDate` (optional): End date filter (YYYY-MM-DD or ISO 8601 format)

Example:

```
GET /audit/?userId=aDzwq3G6IBVRJVCEFijdg7B0fwq2&startDate=2025-09-01&endDate=2025-09-30
```

Response:

```
{
  "entries": [
    {
      "timestamp": "2025-09-23T14:30:15.123Z",
      "userId": "aDzwq3G6IBVRJVCEFijdg7B0fwq2",
      "userEmail": "analyst@example.com",
      "action": "case-export",
      "result": "success",
      "details": {
        "caseNumber": "CASE-2025-001",
        "workflowPhase": "case-export",
        "validationErrors": []
      }
    }
  ],
  "total": 42
}
```

Status Codes: - `200`: Success - `400`: Invalid parameters or missing userId - `403`: Forbidden - `500`: Server error

Query Behavior: - If no date range is specified, returns entries for the current date only - Date range queries read daily audit files and aggregate results - Results are sorted by timestamp in descending order (most recent first) - Additional filtering (action, result, case

number, pagination) is applied client-side in `app/services/audit.service.ts`

Storage Architecture: - Audit entries are stored in R2 with daily file organization: `audit-trails/{userId}/{YYYY-MM-DD}.json` - Immutable entries: Once created, audit entries cannot be modified or deleted - All entries include tamper-proof timestamps from server time

Error Handling

Standard Error Response Format

```
{
  "error": "string"
}
```

Notes: - Most workers return JSON error payloads in the shape above. - `user-worker` and `keys-worker` may also return plain-text error messages for some failure paths.

Common Error Codes

- `Forbidden`: Invalid authentication header or token
- `Method not allowed`: Unsupported HTTP verb for the endpoint
- `User not found`: User UID does not exist in KV
- `Invalid manifest payload`: Signing payload failed schema validation
- `userId parameter is required`: Audit endpoint called without required query parameter

HTTP Status Codes

- `200`: Success
- `201`: Created
- `400`: Bad Request
- `403`: Forbidden
- `404`: Not Found
- `405`: Method Not Allowed
- `500`: Internal Server Error

Type Definitions

Audit Trail Types

ValidationAuditEntry Interface

Core audit entry structure for all validation events:

```
interface ValidationAuditEntry {
  timestamp: string;           // ISO 8601 timestamp
  userId: string;             // User identifier
  userEmail: string;          // User email for identification
  action: AuditAction;        // What action was performed
  result: AuditResult;        // Success/failure/warning/blocked
  details: AuditDetails;      // Action-specific details
}
```

AuditAction Type

All supported audit actions:

```
type AuditAction =
  // Case Management Actions
  | 'case-create' | 'case-rename' | 'case-delete'
  // Confirmation Workflow Actions
  | 'case-export' | 'case-import' | 'confirmation-create'
  | 'confirmation-export' | 'confirmation-import'
  // File Operations
  | 'file-upload' | 'file-delete' | 'file-access'
  // Annotation Operations
  | 'annotation-create' | 'annotation-edit' | 'annotation-delete'
  // User & Session Management
  | 'user-login' | 'user-logout' | 'user-profile-update'
  | 'user-password-reset' | 'user-account-delete'
  // Registration & MFA
  | 'user-registration' | 'email-verification' | 'mfa-enrollment' | 'mfa-a
  // Document Generation
  | 'pdf-generate'
  // Security & Monitoring
  | 'security-violation'
  // Legacy actions (for backward compatibility)
  | 'import' | 'export' | 'confirm' | 'validate';
```

AuditResult Type

Result types for audit operations:

```
type AuditResult = 'success' | 'failure' | 'warning' | 'blocked' | 'pendin
```

AuditDetails Interface

Detailed information for each audit entry:

```
interface AuditDetails {
  // Core identification
  fileName?: string;
  fileType?: AuditFileType;
  caseNumber?: string;
  confirmationId?: string;

  // Validation & Security
  hashValid?: boolean;
  validationErrors: string[];
  securityChecks?: SecurityCheckResults;

  // Context & Workflow
  originalExaminerUid?: string;
  reviewingExaminerUid?: string;
  workflowPhase?: WorkflowPhase;

  // Performance & Metrics
  performanceMetrics?: PerformanceMetrics;

  // Specialized details
  caseDetails?: CaseAuditDetails;
  fileDetails?: FileAuditDetails;
  annotationDetails?: AnnotationAuditDetails;
  sessionDetails?: SessionAuditDetails;
  securityDetails?: SecurityAuditDetails;
  userProfileDetails?: UserProfileAuditDetails;
}
```

AuditTrail Interface

Complete audit trail for a case or workflow:

```
interface AuditTrail {
  caseNumber: string;
  workflowId: string;           // Unique identifier linking related entri
  entries: ValidationAuditEntry[];
  summary: AuditSummary;
}
```

AuditSummary Interface

Summary of audit trail for reporting and compliance:

```
interface AuditSummary {
  totalEvents: number;
  successfulEvents: number;
  failedEvents: number;
  warningEvents: number;
  workflowPhases: WorkflowPhase[];
  participatingUsers: string[];     // User IDs
  startTimestamp: string;
  endTimestamp: string;
  complianceStatus: 'compliant' | 'non-compliant' | 'pending';
  securityIncidents: number;
}
```

WorkflowPhase Type

Workflow phases for tracking different types of forensic activities:

```
type WorkflowPhase =
  | 'casework'           // Case, notes, image, and pdf related actions
  | 'case-export'        // Only case exporting
  | 'case-import'        // Only case importing
  | 'confirmation'       // Only confirmation-related activity
  | 'user-management';   // User login, logout, profile management, accoun
```

Core Annotation Types

AnnotationData Interface

The comprehensive data structure for all annotation information:

```
interface AnnotationData {
  leftCase: string;
  rightCase: string;
  leftItem: string;
  rightItem: string;
  caseFontColor?: string;
  classType?: 'Bullet' | 'Cartridge Case' | 'Other';
  customClass?: string;
  classNote?: string;
  indexType?: 'number' | 'color';
  indexNumber?: string;
  indexColor?: string;
  supportLevel?: 'ID' | 'Exclusion' | 'Inconclusive';
  hasSubclass?: boolean;
  includeConfirmation: boolean;
  confirmationData?: ConfirmationData;
  additionalNotes?: string;
  boxAnnotations?: BoxAnnotation[];
  updatedAt: string;
  earliestAnnotationTimestamp?: string; // ISO timestamp of first annotati
}
```

ConfirmationData Interface

Authenticated confirmation data structure for forensic verification workflow (defined in `app/types/annotations.ts`):

```
interface ConfirmationData {
  fullName: string;           // Confirming examiner's full name
  badgeId: string;            // Badge/ID number of confirming examiner
  timestamp: string;          // Human-readable confirmation timestamp
  confirmationId: string;     // Unique ID generated at confirmation time
  confirmedBy: string;        // User UID of the confirming examiner
  confirmedByEmail: string;   // Email of the confirming examiner
  confirmedByCompany: string; // Company/Lab of the confirming examiner
  confirmedAt: string;        // ISO timestamp of confirmation
}
```

Purpose: Captures authenticated digital confirmation from reviewing examiners for independent verification of forensic conclusions.

Workflow Context: - Created when reviewing examiner confirms findings using the Confirmation modal - Stored within AnnotationData when `includeConfirmation` is true - Exported via case export and imported by original examiner - Includes unique confirmation ID for audit trails and traceability - Appears in PDF reports with full examiner attribution

BoxAnnotation Interface

Individual box annotation structure with percentage-based coordinates (defined in `app/types/annotations.ts`):

```
interface BoxAnnotation {
  id: string;
  x: number;         // Percentage 0-100
  y: number;         // Percentage 0-100
  width: number;     // Percentage 0-100
  height: number;    // Percentage 0-100
  color: string;     // Hex color code
  label?: string;    // Optional label text
  timestamp: string; // Creation timestamp (ISO 8601 format)
}
```

User Management Types

UserData Interface

Core user profile and permissions structure:

```typescript
interface UserData {
  uid: string;
  email: string | null;
  firstName: string;
  lastName: string;
  company: string;
  permitted: boolean;
  cases: Array<{
    caseNumber: string;
    createdAt: string;
  }>;
  readOnlyCases?: Array<{
    caseNumber: string;
    importedAt: string;
    originalExportDate: string;
    originalExportedBy: string;
    sourceHash?: string;
    sourceManifestVersion?: string;
    sourceSignatureKeyId?: string;
    sourceSignatureValid?: boolean;
    isReadOnly: true;
  }>;
  createdAt: string;
  updatedAt?: string;
}
```

UserLimits Interface

User account limitations and quotas:

```typescript
interface UserLimits {
  maxCases: number;
  maxFilesPerCase: number;
}
```

File Management Types

FileData Interface

Core file information structure:

```typescript
interface FileData {
  id: string;
  originalFilename: string;
  uploadedAt: string;
}
```

FileUploadResponse Interface

Cloudflare Images API response structure:

```typescript
interface FileUploadResponse {
  success: boolean;
  result: {
    id: string;
    filename: string;
    uploaded: string;
    requireSignedURLs: boolean;
    variants: string[];
  };
  errors: Array<{
    code: number;
    message: string;
  }>;
  messages: string[];
}
```

ImageUploadResponse Interface

Image upload response wrapper (extends FileUploadResponse):

```
interface ImageUploadResponse {
  success: boolean;
  result: FileUploadResponse['result'];
  errors: FileUploadResponse['errors'];
  messages: FileUploadResponse['messages'];
}
```

Case Management Types

CaseData Interface

Case structure with associated files:

```
interface CaseData {
  createdAt: string;
  caseNumber: string;
  files: FileData[];
}
```

CaseActionType

Case operation type definition for UI state management:

```
type CaseActionType = 'loaded' | 'created' | 'deleted' | null;
```

CasesToDelete Interface

Structure for batch case deletion operations:

```
interface CasesToDelete {
  casesToDelete: string[];
}
```

Case Export Types

CaseExportData Interface

Complete case export data structure for single case exports:

```typescript
interface CaseExportData {
  metadata: {
    caseNumber: string;
    caseCreatedDate: string;
    exportDate: string;
    exportedBy: string | null;
    exportedByUid: string;
    exportedByName: string;
    exportedByCompany: string;
    striaeExportSchemaVersion: string;
    totalFiles: number;
  };
  files: Array<{
    fileData: FileData;
    annotations?: AnnotationData;
    hasAnnotations: boolean;
  }>;
  summary?: {
    filesWithAnnotations: number;
    filesWithoutAnnotations: number;
    totalBoxAnnotations: number;
    filesWithConfirmations?: number;
    filesWithConfirmationsRequested?: number;
    lastModified?: string;
    earliestAnnotationDate?: string;
    latestAnnotationDate?: string;
    exportError?: string;
  };
}
```

AllCasesExportData Interface

Comprehensive export data structure for bulk case exports:

```typescript
interface AllCasesExportData {
  metadata: {
    exportDate: string;
    exportedBy: string | null;
    exportedByUid: string;
    exportedByName: string;
    exportedByCompany: string;
    striaeExportSchemaVersion: string;
    totalCases: number;
    totalFiles: number;
    totalAnnotations: number;
    totalConfirmations: number;
    totalConfirmationsRequested: number;
  };
  cases: CaseExportData[];
  summary?: {
    casesWithFiles: number;
    casesWithAnnotations: number;
    casesWithoutFiles: number;
    lastModified?: string;
    earliestAnnotationDate?: string;
    latestAnnotationDate?: string;
  };
}
```

ExportFormat Type

Export format values used by `ExportOptions.format`:

```typescript
type ExportFormat = 'json' | 'csv';
```

ExportOptions Interface

Configuration options for case export operations:

```
interface ExportOptions {
  format?: 'json' | 'csv';
  includeMetadata?: boolean;
  includeUserInfo?: boolean;
  protectForensicData?: boolean;
}
```

Export Features:

- **Format Support**: JSON, CSV/Excel, and ZIP export formats
- **ZIP Export with Images**: Single case exports can include associated image files packaged in ZIP format
- **Comprehensive Data**: All annotation fields including case identifiers, colors, classifications, and box annotations
- **Bulk Operations**: Export all cases with progress tracking and error handling
- **Excel Multi-Worksheet**: CSV format creates Excel files with multiple worksheets for bulk exports
- **Split Box Annotations**: Each box annotation gets its own row in CSV/Excel for better data analysis
- **Metadata Rich**: Complete export metadata including timestamps, user info, and summary statistics
- **Error Handling**: Graceful handling of failed case exports with detailed error information
- **Progress Tracking**: Real-time progress updates for ZIP generation and image downloads

Case Import Types

CaseImportProps Interface

Props interface for the main case import component:

```
interface CaseImportProps {
  isOpen: boolean;
  onClose: () => void;
  onImportComplete?: (result: ImportResult | ConfirmationImportResult) =>
}
```

ImportOptions Interface

Configuration options for case import operations:

```
interface ImportOptions {
  overwriteExisting?: boolean;
  validateIntegrity?: boolean;
  preserveTimestamps?: boolean;
}
```

ImportResult Interface

Result structure returned from case import operations:

```
interface ImportResult {
  success: boolean;
  caseNumber: string;
  isReadOnly: boolean;
  filesImported: number;
  annotationsImported: number;
  errors?: string[];
  warnings?: string[];
}
```

ConfirmationImportResult Interface

Result structure returned from confirmation import operations:

```
interface ConfirmationImportResult {
  success: boolean;
  caseNumber: string;
  confirmationsImported: number;
  imagesUpdated: number;
  errors?: string[];
  warnings?: string[];
}
```

ReadOnlyCaseMetadata Interface

Metadata structure for imported read-only cases:

```
interface ReadOnlyCaseMetadata {
  caseNumber: string;
  importedAt: string;
  originalExportDate: string;
  originalExportedBy: string;
  sourceHash?: string;
  sourceManifestVersion?: string;
  sourceSignatureKeyId?: string;
  sourceSignatureValid?: boolean;
  isReadOnly: true;
}
```

CaseImportPreview Interface

Preview information generated during ZIP parsing before import:

```
interface CaseImportPreview {
  caseNumber: string;
  exportedBy: string | null;
  exportedByName: string | null;
  exportedByCompany: string | null;
  exportDate: string;
  totalFiles: number;
  caseCreatedDate?: string;
  hashValid?: boolean;
  hashError?: string;
  hasAnnotations: boolean;
  validationSummary: string;
  errors?: string[];
  validationDetails?: {
    hasForensicManifest: boolean;
    dataValid?: boolean;
    imageValidation?: { [filename: string]: boolean };
    manifestValid?: boolean;
    signatureValid?: boolean;
    signatureKeyId?: string;
    signatureError?: string;
    validationSummary?: string;
    integrityErrors?: string[];
  };
}
```

Import Features:

- **Complete ZIP Package Import**: Full case data and image import from exported ZIP packages
- **Read-Only Protection**: Imported cases are automatically set to read-only mode for secure review
- **Duplicate Prevention**: Prevents import if user was the original case analyst
- **Progress Tracking**: Multi-stage progress reporting with detailed status updates

- **Image Integration**: Automatic upload and association of all case images
- **Metadata Preservation**: Complete preservation of original export metadata and timestamps
- **Data Integrity**: Comprehensive validation of ZIP contents and case data structure

CSV/Excel Export Details:

- **Single Case CSV**: Comprehensive CSV with individual rows for each box annotation
- **Bulk Export Excel**: Multi-worksheet Excel file with summary sheet and individual case sheets
- **Complete Data Parity**: CSV/Excel exports contain identical data to JSON exports
- **Enhanced Box Annotations**: Includes coordinates, colors, and timestamps (label removed)
- **Classification Data**: Full support for case identifiers, color schemes, and forensic classifications
- **Optimized Layout**: File metadata appears only on first row per file, subsequent rows contain only box data

ZIP Export Functionality:

- **Single Case Only**: ZIP exports are restricted to individual cases to manage file size
- **Image Inclusion**: All associated image files packaged with original filenames preserved
- **Data File**: Selected format (JSON or CSV) included in ZIP package
- **README Generation**: Automatic README.txt with case summary and statistics
- **Progress Feedback**: Real-time progress updates during ZIP creation and image downloads
- **Error Recovery**: Graceful handling of individual image download failures
- **JSZip Integration**: Browser-based ZIP generation using JSZip library

Centralized Type Management

Type Definition Location: All interfaces are centrally defined in `app/types/` with barrel exports from `app/types/index.ts`:

```
// Barrel export structure
export * from './annotations';  // AnnotationData, BoxAnnotation
export * from './user';        // UserData, UserLimits
export * from './file';        // FileData, FileUploadResponse, ImageUploa
export * from './case';        // CaseData, CaseActionType, CaseExportData
export * from './import';      // ImportResult, ConfirmationImportResult,
export * from './export';      // ExportOptions
export * from './audit';       // ValidationAuditEntry, AuditTrail, AuditQ
```

Usage Patterns:

- **Centralized Imports**: `import { UserData, FileData, AnnotationData } from '~/types';`
- **Component Props**: All component interfaces extend or use centralized types
- **Worker APIs**: Type validation based on shared interface definitions
- **PDF Generation**: Type-safe data flow from frontend through workers
- **Data Storage**: Structured data persistence with comprehensive type validation

Benefits:

- Single source of truth for all data structures
- Type safety across entire application stack
- Consistent API contracts between frontend and workers
- Easier refactoring and maintenance with centralized type management
- Compile-time error detection for data structure changes
- Comprehensive coverage of all application domains (user, file, case, annotation)
- Utility types for common patterns (loading states, API responses, pagination)

SDK Examples

JavaScript/TypeScript Client

```
class StriaeAPI {
  constructor(private apiKey: string, private baseUrl: string) {}

  async getUser(userUid: string): Promise<UserData> {
    const response = await fetch(`${this.baseUrl}/${userUid}`, {
      headers: {
        'X-Custom-Auth-Key': this.apiKey,
        'Content-Type': 'application/json'
      }
    });

    if (!response.ok) {
      throw new Error(`API Error: ${response.status}`);
    }

    return response.json();
  }

  async uploadImage(file: File): Promise<UploadResult> {
    const formData = new FormData();
    formData.append('file', file);

    const response = await fetch(`${this.baseUrl}`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${this.apiKey}`
      },
      body: formData
    });

    return response.json();
  }
}
```

Error Handling Example

```
try {
  const userData = await api.getUser(userUid);
  console.log('User data:', userData);
} catch (error) {
  if (error.status === 404) {
    console.log('User not found');
  } else if (error.status === 403) {
    console.log('Authentication failed (forbidden)');
  } else {
    console.error('API error:', error);
  }
}
```